

# Exam2023

June 20, 2026

Denne skabelon indeholder fire sektioner: **Opgave 1** til **Opgave 4**. Hver sektion har fire under-spørgsmål: **Spørgsmål 1** til **Spørgsmål 4**. En notebook konverteres til pdf ved at køre: `jupyter nbconvert -to pdf ExamTemplate2026.ipynb` i terminalen. Når notebooken eksporteres/printes til PDF, starter hver ny opgave på en ny side.

## 0.1 Kopiérbar blok til indsættelse af billeder

### 0.1.1 Markdown-metode

Kopiér linjen nedenfor til en Markdown-celle og ret filnavn/størrelse efter behov:

```

```

### 0.1.2 Python-metode

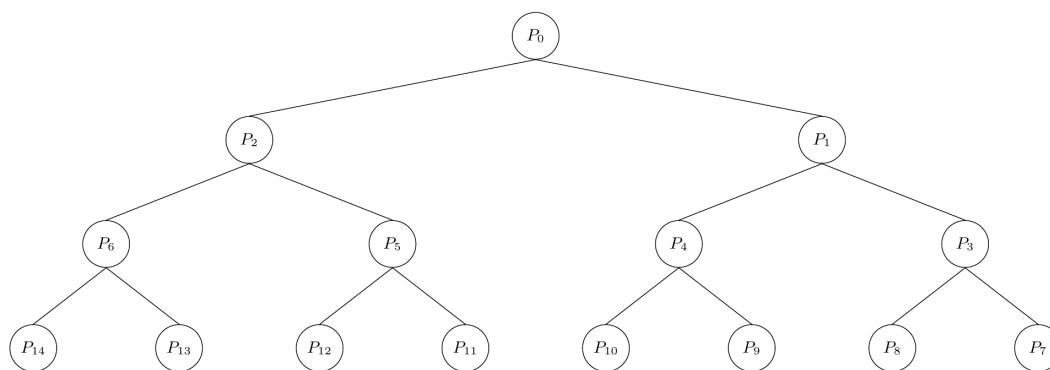
Kør kodecellen nedenfor, og brug funktionen `indsæt_billede(...)` i dine svar.

```
[17]: from IPython.display import Image, display

def indsæt_billede(sti, bredde=600):
    """Viser et billede i notebooken.

    Parametre:
    sti: Filsti til billedet, fx 'images/figur1.png'
    bredde: Billedets bredde i pixels, fx 600
    """
    display(Image(filename=sti, width=bredde))

indsæt_billede('tree.png', bredde=600)
```



```
[18]: # Imports
import numpy as np
import pulp as PLP
```

```
import matplotlib.pyplot as plt
```

## 1 Opgave 1

I model this as set covering problem. I have implemented a base model for such problems in the class below

```
[19]: class set_covering_problem:
    """ This implementaion follows Set Covering, Packing, Partition from Uge
    ↪13, it is modified to be
    able to handle the weighted problem, by allowing weights != 1 for the
    ↪constraints
    Input:
    adj_matrix: Adjacency matrix shape(m,n), that to some criteria defines if i
    ↪= 1...m'th element is
    contained in the j = 1...n subset, thus adj_matrix[i][j] = 1 if i is in
    ↪subset j, and 0 otherwise

    weights: A list of length m, where weights[i] is the weight of the i'th
    ↪element, allowing
    requiring 1 or more subsets to cover the element, thus weights[i] = 1 means
    ↪that the i'th element must be covered by
    at least one subset, weights[i] = 2 means that the i'th element must be
    ↪covered by at least two subsets, and so on.

    costs: A list of length n, where costs[j] is the cost of including the j'th
    ↪subset in the solution.

    F: the Familly of possible subsets
    """
    def __init__(self, adj_matrix, weights, costs, F):
        self.adj_matrix = adj_matrix
        self.weights = weights
        self.costs = costs
        self.m , self.n = adj_matrix.shape
        self.F = F

        self.model = PLP.LpProblem(name="SetCoveringProblem", sense=PLP.
    ↪LpMinimize)
        self.delta = PLP.LpVariable.dicts("delta", range(self.n), cat=PLP.
    ↪LpBinary, lowBound=0, upBound=1)

        # Add objective
        self.model += PLP.lpSum(self.costs[j] * self.delta[j] for j in
    ↪range(self.n)), "Objective"
        self.constraints = "NOT ADDED"
```

```

def construct_constraints(self):
    for i in range(self.m):
        self.model += PLP.lpSum(self.adj_matrix[i][j]*self.delta[j]
                                for j in range(self.n)) >= self.weights[i],
    f"Subset_Constraint_{i}"
    self.constraints = "ADDED"

def solve_and_print(self, quiet = True):
    if self.constraints != "ADDED":
        raise ValueError("You must add the constraints before solving")
    eps = 0.01
    self.model.solve(PLP.PULP_CBC_CMD(msg=0 if quiet else 1))
    print("Status:", PLP.LpStatus[self.model.status])
    print("Objective value:", PLP.value(self.model.objective))
    for i in range(self.n):
        if self.delta[i].varValue is not None and self.delta[i].varValue >
    0 + eps:
        print(f"Subset {self.F[i]} is included in the solution with
    cost {self.costs[i]}")

    ### PLOTTING CODE ###

def get_selected_subset_indices(self, eps=1e-6):
    """Return indices of subsets selected in the solved model."""
    return [j for j in range(self.n) if self.delta[j].varValue is not None
    and self.delta[j].varValue > eps]

def plot_solution(
    self,
    element_coordinates,
    subset_coordinates=None,
    selected_indices=None,
    coverage_radius=None,
    annotate_elements=True,
    annotate_subsets=False,
    show_all_subsets=False,
    figsize=(8, 8),
    title="Set Covering Solution",
    equal_axis=True,
):
    """
    Plot element coordinates and selected subset coordinates.

    Args:

```

```

        element_coordinates: Sequence of (x, y) coordinates for elements to
        ↪cover (size m).
        subset_coordinates: Sequence of (x, y) coordinates for subsets
        ↪(size n). Defaults to self.F.
        selected_indices: Optional explicit subset indices to plot as
        ↪selected.
        coverage_radius: Optional radius for drawing coverage circles
        ↪around selected subsets.
    """
    if subset_coordinates is None:
        subset_coordinates = self.F

    if len(element_coordinates) != self.m:
        raise ValueError(f"Expected {self.m} element coordinates, got
        ↪{len(element_coordinates)}")
    if len(subset_coordinates) != self.n:
        raise ValueError(f"Expected {self.n} subset coordinates, got
        ↪{len(subset_coordinates)}")

    if selected_indices is None:
        selected_indices = self.get_selected_subset_indices()
        if len(selected_indices) == 0:
            raise ValueError("No selected subsets found. Solve the model
            ↪before plotting.")

    for idx in selected_indices:
        if idx < 0 or idx >= self.n:
            raise IndexError(f"Selected subset index {idx} is out of bounds
            ↪for n={self.n}")

    element_x = [coord[0] for coord in element_coordinates]
    element_y = [coord[1] for coord in element_coordinates]
    selected_coords = [subset_coordinates[j] for j in selected_indices]
    selected_x = [coord[0] for coord in selected_coords]
    selected_y = [coord[1] for coord in selected_coords]

    fig, ax = plt.subplots(figsize=figsize)

    if coverage_radius is not None:
        if coverage_radius <= 0:
            raise ValueError("coverage_radius must be positive")
        for idx, coord in enumerate(selected_coords):
            circle = plt.Circle(
                coord,
                coverage_radius,
                color="red",

```

```

        fill=True,
        alpha=0.4,
        label="Coverage Area" if idx == 0 else None,
    )
    ax.add_patch(circle)

plt.scatter(element_x, element_y, color="blue", label="Elements")

if annotate_elements:
    for i, (x_coord, y_coord) in enumerate(element_coordinates):
        plt.text(x_coord + 0.1, y_coord + 0.1, str(i + 1), fontsize=9,
↪color="blue")

if show_all_subsets:
    all_subset_x = [coord[0] for coord in subset_coordinates]
    all_subset_y = [coord[1] for coord in subset_coordinates]
    plt.scatter(all_subset_x, all_subset_y, color="lightgray",
↪marker="x", label="All Subsets")

plt.scatter(selected_x, selected_y, color="green", label="Selected_
↪Subsets")

if annotate_subsets:
    for idx, (x_coord, y_coord) in zip(selected_indices,
↪selected_coords):
        plt.text(x_coord + 0.1, y_coord + 0.1,
↪str(subset_coordinates[idx]), fontsize=9, color="red")

plt.title(title)
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.legend()
if equal_axis:
    plt.axis("equal")
plt.grid()
plt.show()
return fig, ax

```

## 1.1 Spørsmål 1

We first model this as an equal and equal weight problem. We need define an adjacency matrix based on the  $L_2$  distance, note that every city included in the solution belongs to the family of subsets  $F$

```

[39]: # Define adj. matrix
x = "2.5 3.7 1.9 3.1 4.5 6.2 7.2 7.7 2.2 2.8 5.1 6.5 5.4 4.1 4.5".split(" ")
x = [float(i) for i in x]

y = "7.5 7.7 4.7 3.5 4.5 2.1 2.8 1.4 3.6 4.2 6.1 5.3 5.0 2.7 3.5".split(" ")
y = [float(i) for i in y]

F = list(zip(x,y))
S = F
def adj_from_coordinates(S, F, L):
    """
    :param S: Input coordinates
    :param F: Possible coordinates where we can place things
    Note S and F can be equal
    :return:
    adj. matrix
    """
    dist = lambda p, q : ((p[0] - q[0])**2 + (p[1] - q[1])**2)**0.5
    adj = np.zeros((len(S),len(F)))
    for i in range(len(S)):
        for j in range(len(F)):
            adj[i][j] = 1 if dist(S[i], F[j]) <= L else 0
    return adj

adj_matrixL2 = adj_from_coordinates(S, F , 2)
weights = [1]*len(S)
costs = [1]*len(S)

SCPL2 = set_covering_problem(adj_matrixL2, weights, costs, F)
SCPL2.construct_constraints()
SCPL2.solve_and_print()
SCPL2.plot_solution(element_coordinates=S, subset_coordinates=F,
    ↪coverage_radius= 2)
plt.show()

```

Status: Optimal

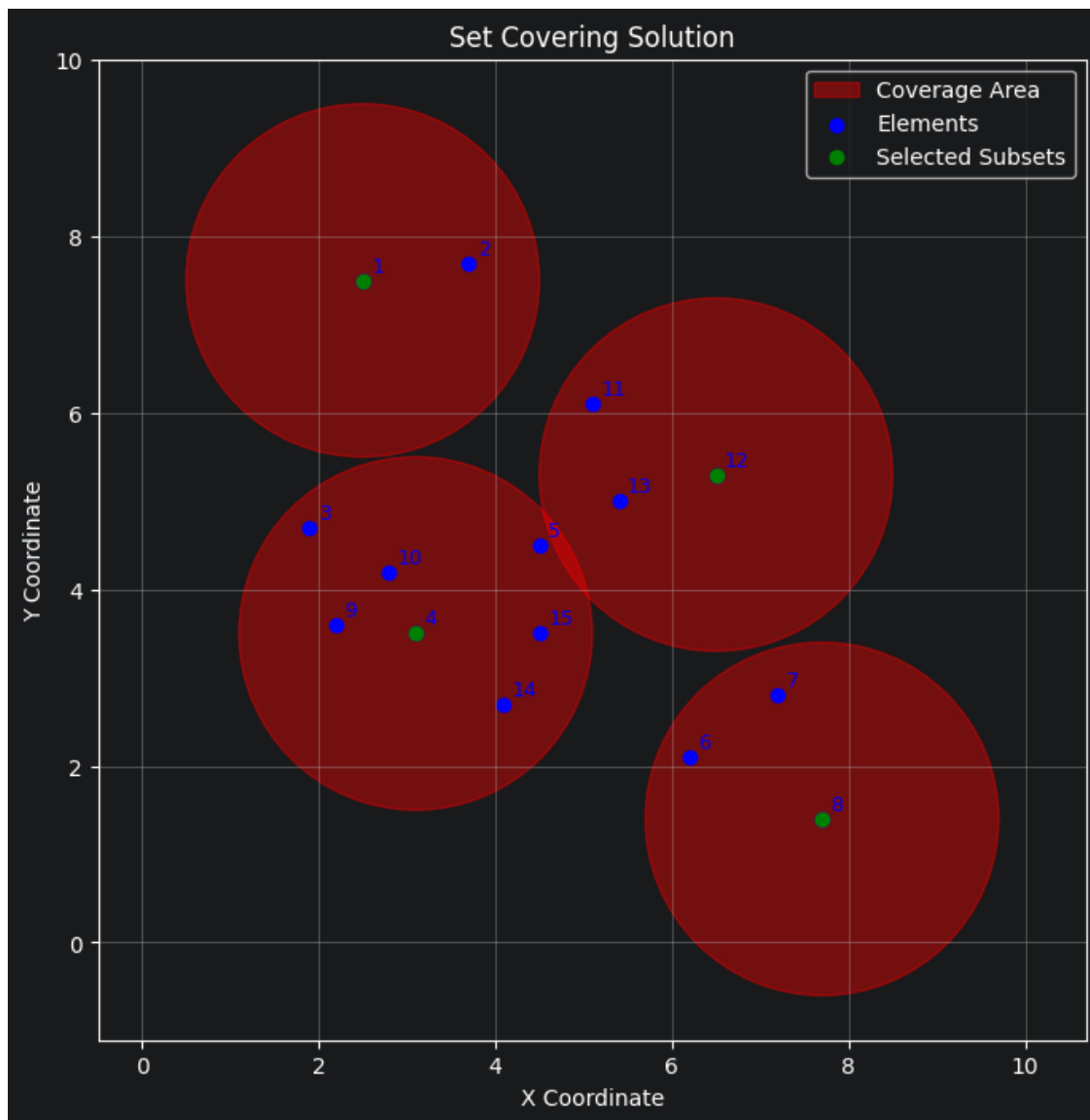
Objective value: 4.0

Subset (2.5, 7.5) is included in the solution with cost 1

Subset (3.1, 3.5) is included in the solution with cost 1

Subset (7.7, 1.4) is included in the solution with cost 1

Subset (6.5, 5.3) is included in the solution with cost 1



In similar fashion a solution for  $L = 3$  is computed

```
[40]: adj_matrixL3 = adj_from_coordinates(S, F , 3)
weights = [1]*len(S)
costs = [1]*len(S)

SCPL3 = set_covering_problem(adj_matrixL3, weights, costs, F)
SCPL3.construct_constraints()
SCPL3.solve_and_print()
SCPL3.plot_solution(element_coordinates=S, subset_coordinates=F,
    ↪coverage_radius= 3)
plt.show()
```



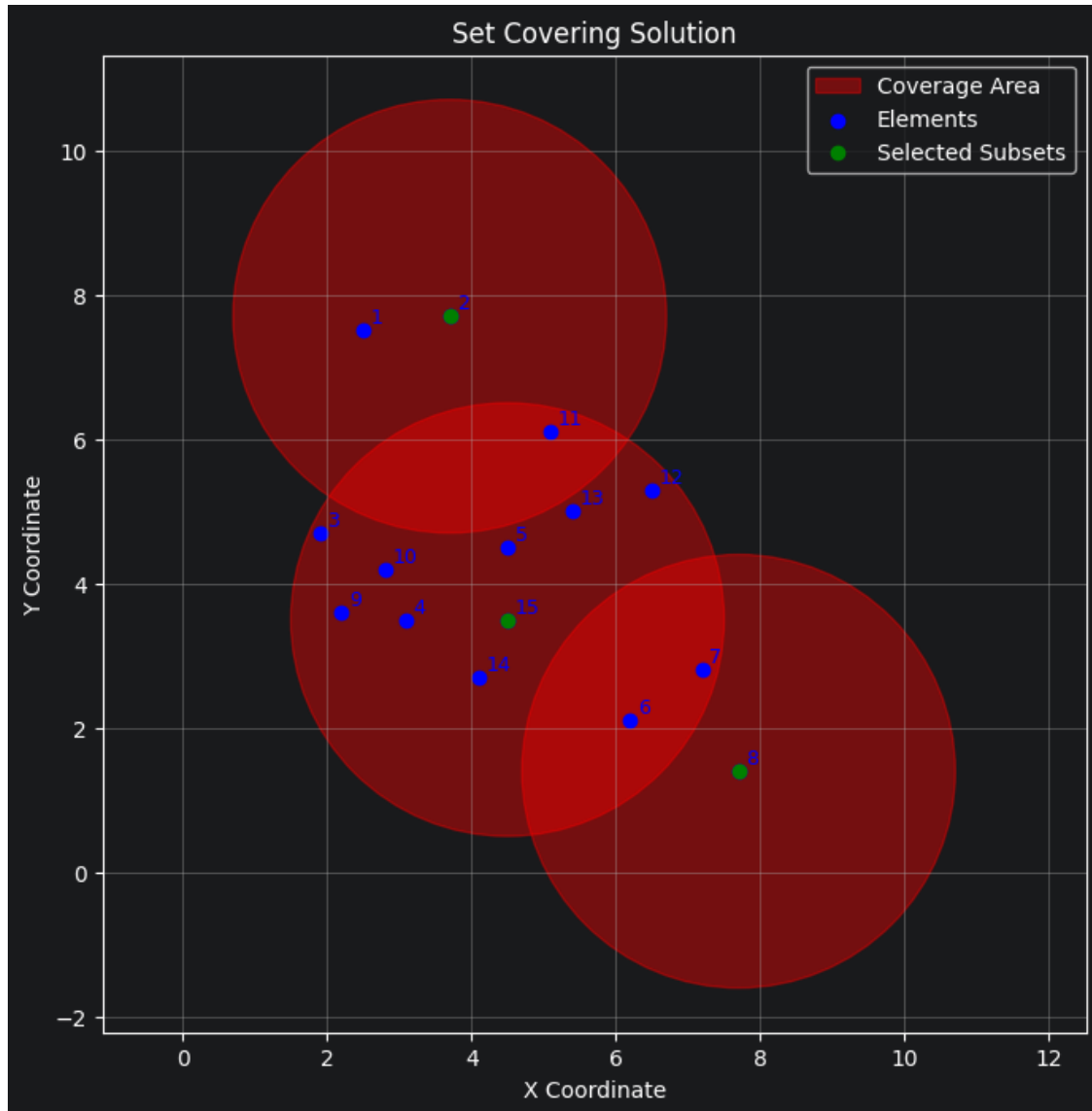
Status: Optimal

Objective value: 3.0

Subset (3.7, 7.7) is included in the solution with cost 1

Subset (7.7, 1.4) is included in the solution with cost 1

Subset (4.5, 3.5) is included in the solution with cost 1



## 1.2 Spørgsmål 2

We can solve this by adding a weight of 2 to the first 8 cities for each coverage radius

```
[42]: adj_matrixL2 = adj_from_coordinates(S, F , 2)
weights = [2]*8 + [1]*(len(S)-8)
costs = [1]*len(S)
```

```
SCPL2 = set_covering_problem(adj_matrixL2, weights, costs, F)
SCPL2.construct_constraints()
SCPL2.solve_and_print()
SCPL2.plot_solution(element_coordinates=S, subset_coordinates=F,
    ↪coverage_radius= 2)
plt.show()
```

Status: Optimal

Objective value: 7.0

Subset (2.5, 7.5) is included in the solution with cost 1

Subset (3.7, 7.7) is included in the solution with cost 1

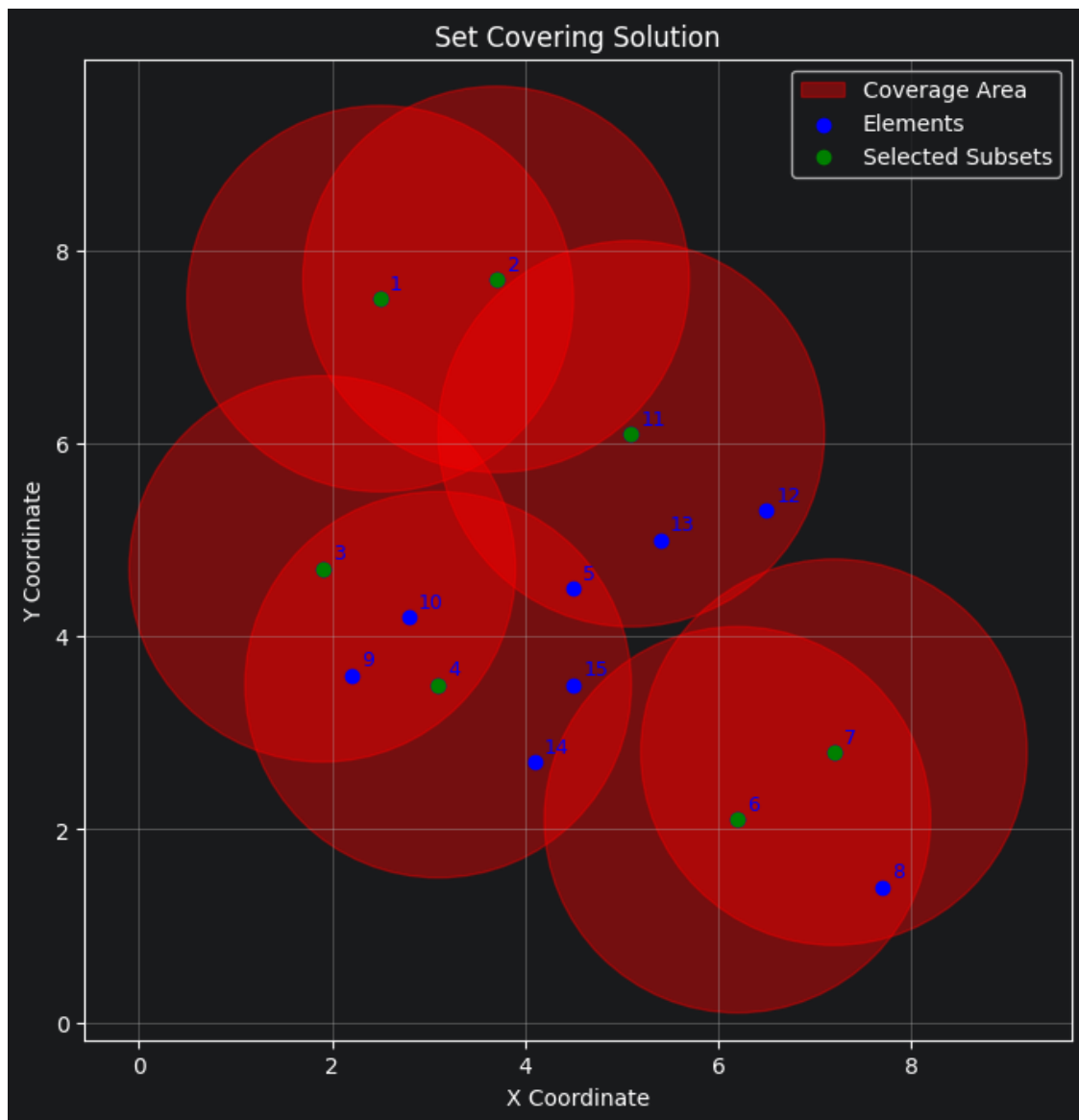
Subset (1.9, 4.7) is included in the solution with cost 1

Subset (3.1, 3.5) is included in the solution with cost 1

Subset (6.2, 2.1) is included in the solution with cost 1

Subset (7.2, 2.8) is included in the solution with cost 1

Subset (5.1, 6.1) is included in the solution with cost 1



and for  $L = 3$

```
[43]: adj_matrixL3 = adj_from_coordinates(S, F , 3)
weights = [2]*8 + [1]*(len(S)-8)
costs = [1]*len(S)

SCPL3 = set_covering_problem(adj_matrixL3, weights, costs, F)
SCPL3.construct_constraints()
SCPL3.solve_and_print()
SCPL3.plot_solution(element_coordinates=S, subset_coordinates=F,
    ↪ coverage_radius= 3)
plt.show()
```

Status: Optimal

Objective value: 6.0

Subset (2.5, 7.5) is included in the solution with cost 1

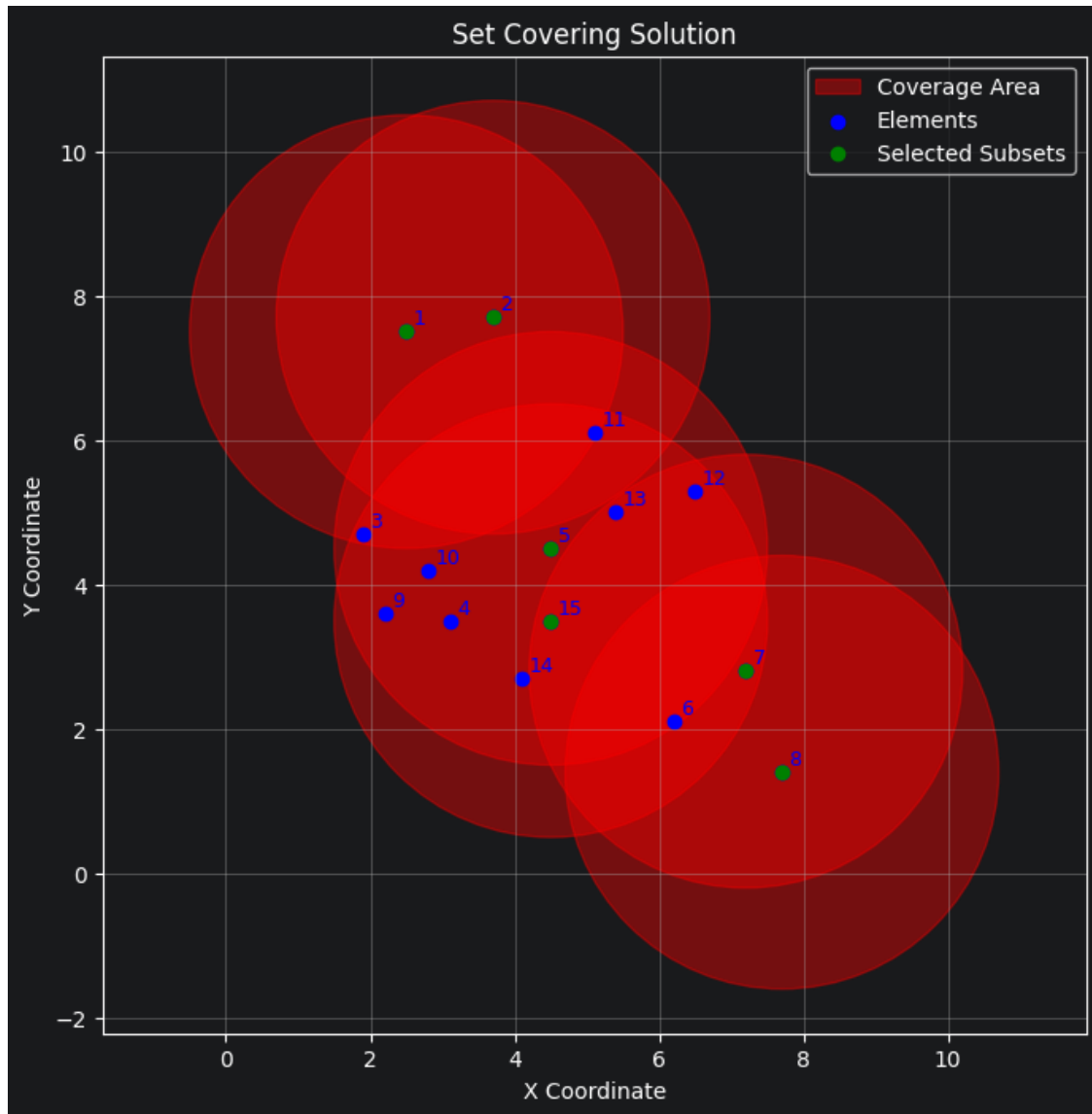
Subset (3.7, 7.7) is included in the solution with cost 1

Subset (4.5, 4.5) is included in the solution with cost 1

Subset (7.2, 2.8) is included in the solution with cost 1

Subset (7.7, 1.4) is included in the solution with cost 1

Subset (4.5, 3.5) is included in the solution with cost 1



### 1.3 Spørsmål 3

This can be done by including the constraint

$$\delta_1 + \delta_5 + \delta_8 \leq 2$$

Such that at most 2 of the 3 cities are used.

[22]: *# Beregninger/kode til dette spørgsmål kan skrives her*

#### 1.4 Spørgsmål 4

We want the following relation

$$\delta_1 + \delta_5 = 2 \implies \delta_8 = 1$$

We can do this by adding the constraint

$$\delta_1 + \delta_5 \leq 1 + \delta_8$$

[23]: *# Beregninger/kode til dette spørgsmål kan skrives her*

## 2 Opgave 2

I have implemented the code for the Gavish graves CVRP below. Explanation are stated as comments in the code. The CVRP is a generalization of the ATSP, and is equivalent if  $m = 1$  and  $Q = n - 1$

```
[56]: class cvrp_gavish_graves:
    """
    This implementation follows from uge 13 VRP-MTZ, and defines the
    ↪Capacitated Vehicle Routing Problem (CVRP) using
    the MTZ formulation.

    Punkt 1 repræsenterer et depot, og hvert af de øvrige punkter repræsenterer
    ↪en kunde
    I Til hver kant  $(i, j)$  er tilknyttet en omkostning  $c_{ij}$ 
    I Der er endvidere oplyst et antal vogne  $m$ , som alle er placeret i depotet
    I Vognene er identiske og har hver en lastekapacitet  $Q$ 
    I For hver kunde  $i = 2, \dots, n$  er oplyst en efterspørgsel  $q_i$ , som skal
    ↪leveres fra depotet til kunden
    I Problemet går ud på at bestemme  $m$  ruter, som har minimal samlet
    ↪omkostning, under flg.
    begrænsninger:
    I Hver rute starter og slutter i depotet
    I Hver kunde bliver besøgt netop én gang, hvor hele den efterspurgte
    ↪mængde leveres
    I Den samlede mængde på hver rute må ikke overstige vognekapaciteten

    """

    def __init__(self, m, n, demand, vehicle_capacity, cost_matrix,
    ↪x_coords=None, y_coords=None):
        # Number of vertices
        self.n = n
        # Number of travellers
        self.m = m
        # Demand for each vertex,  $q_i$ .
        self.demand = demand
        # Maximum, identical vehicle capacity
        self.vehicle_capacity = vehicle_capacity

        self.cost_matrix = cost_matrix

        # If no alternative cost matrix is given, compute the L2-distance cost.
        if x_coords is not None and y_coords is not None and cost_matrix is
    ↪None:
            self.cost_matrix = self.cost_from_coordinates(x_coords, y_coords)
```

```

# ILP problem
self.model = PLP.LpProblem(name="MultipleTSP", sense=PLP.LpMinimize)
# Define arcs,
self.arcs = [(i, j) for i in range(n) for j in range(n) if i != j]

#### Variable definition ####

# Binary variable indicating whether arc (i,j) is used in the solution
self.x = PLP.LpVariable.dicts("x", self.arcs, lowBound=0, upBound=1,
↪cat=PLP.LpBinary)
self.f = PLP.LpVariable.dicts("f", self.arcs, lowBound=0, cat = PLP.
↪LpContinuous)
# Indicator variable that indentifies the i'th vertex position in the
↪traversal of the rute
self.u = PLP.LpVariable.dicts("u", range(n), lowBound=0)

#### Obejctive function ####
self.model += PLP.lpSum(self.cost_matrix[i][j] * self.x[(i, j)] for i,
↪j in self.arcs), "Objective"

def define_constraints(self):
    #### Constraints ####

    # We must have m routes leaving the depot (vertex 0)
    self.model += PLP.lpSum(self.x[(0, j)] for j in range(1, self.n)) ==
↪self.m, "LeavingDepot"

    # We must have m routes returning to the depot (vertex 0)
    self.model += PLP.lpSum(self.x[(i, 0)] for i in range(1, self.n)) ==
↪self.m, "EnteringDepot"

    # Outflow is 1 for all vertex
    for i in range(1, self.n):
        self.model += PLP.lpSum(self.x[(i, j)] for j in range(self.n) if
↪(i, j) in self.arcs) == 1, f"Outflow{i}"

    # Inflow is 1 for all vertex
    for j in range(1, self.n):
        self.model += PLP.lpSum(self.x[(i, j)] for i in range(self.n) if
↪(i, j) in self.arcs) == 1, f"Inflow{j}"
    # Amount out = Amount in + demand
    for i in range(1, self.n):
        self.model += (PLP.lpSum(self.f[(i,j)] - self.f[(j,i)]
↪for j in range(self.n)

```

```

        if ((i, j) in self.arcs) or ((j, i) in self.arcs)))
            - self.demand[i] == 0, f"InOutDemand{i}")

    # Bounds for f variables
    for arc in self.arcs:
        self.model += self.f[arc] >= 0, f"LowBoundf{arc}"
        self.model += self.f[arc] <= self.vehicle_capacity * self.x[arc],
        f"HighBoundf{arc}"

    self.constraints = "Added"

def cost_from_coordinates(self, x_cords, y_cords):
    # Computes L2 distance between points, this can serve as a cost.
    n = len(x_cords)
    cost_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i == j:
                # Set diagonal to very large number
                very_large_number = 10 ** 6
                cost_matrix[i][j] = very_large_number
            else:
                diff_x = x_cords[i] - x_cords[j]
                diff_y = y_cords[i] - y_cords[j]
                dist = np.sqrt(diff_x ** 2 + diff_y ** 2)
                cost_matrix[i][j] = dist
    return cost_matrix

def solve_and_print(self, one_indexed = True, msg = False):
    self.model.solve(PLP.PULP_CBC_CMD(msg=msg))
    if self.constraints != "Added":
        raise Exception("You must define the constraints before solving,
        ↳call define_constraints() method")
    print("Status:", PLP.LpStatus[self.model.status])
    eps = 0.5
    for i, j in sorted(
        self.arcs,
        key=lambda arc: self.u[arc[0]].varValue if self.u[arc[0]] else
        ↳1 # Sort according to the u value
    ):
        if self.x[(i, j)].varValue > 0 + eps:
            # Track positioning
            f_step = int((self.f[(i,j)].varValue if self.f[(i,j)].varValue
            ↳is not None else -1) + (1 if one_indexed else 0))

```



```

        # Bool to check if depot
        dep_or_vertex = f"vertex_{j} + (1 if one_indexed else 0)" if j !
↪= 0 else "Depot"
        print(
            f"Arc ({i} + (1 if one_indexed else 0)}, {j} + (1 if
↪one_indexed else 0)})"
            f" is used in the solution with cost {round(self.
↪cost_matrix[i][j], 2)},"
            f" {dep_or_vertex} is visited as the "
            f"{f_step}"
            f" th vertex in the route\n"
        )
        print("Total cost:", PLP.value(self.model.objective))

```

## 2.1 Spørgsmål 1

We get the coordinates for the first 8 cities and. We assume that we have 1 vehicle with a capacity of 8, and each city has a demand of 1.

```

[57]: x_coords = x[0:8]
      y_coords = y[0:8]
      # Number of vehicles
      m = 1
      # Numbe of vertices
      n = 8
      # Vertex demand is one for
      demand = [1]*n
      # vehicle capacity
      Q = n - 1

      ATSP = cvrp_gavish_graves(m, n, demand, Q, cost_matrix= None, x_coords =
↪x_coords, y_coords = y_coords)
      ATSP.define_constraints()
      ATSP.solve_and_print(one_indexed = True)

```

Status: Optimal

Arc (1, 3) is used in the solution with cost 2.86, vertex\_3 is visited as the 1 th vertex in the route

Arc (2, 1) is used in the solution with cost 1.22, Depot is visited as the 8 th vertex in the route

Arc (3, 4) is used in the solution with cost 1.7, vertex\_4 is visited as the 2 th vertex in the route

Arc (4, 6) is used in the solution with cost 3.4, vertex\_6 is visited as the 3 th vertex in the route

Arc (5, 2) is used in the solution with cost 3.3, vertex\_2 is visited as the 7th vertex in the route

Arc (6, 8) is used in the solution with cost 1.66, vertex\_8 is visited as the 4th vertex in the route

Arc (7, 5) is used in the solution with cost 3.19, vertex\_5 is visited as the 6th vertex in the route

Arc (8, 7) is used in the solution with cost 1.49, vertex\_7 is visited as the 5th vertex in the route

Total cost: 18.809640401561047

## 3 Opgave 3

### 3.1 Spørgsmål 1

We can use the network model clustering from shortest path to solve this issue. I have defined a class for the shortest path problem below. This problem is alike the problem NET-2.

```
[58]: class shortest_path:
    """
    This implementation of the shortest path problem, follows from Uge 7 Afsnit 5.3
    ↪ 5.3 Netværksmodeller.

    Input:
    nodes - list of nodes
    edges - list of edges
    cost - the cost associated with each edge (i,j)
    start_node - the index of the starting node
    end_node - the index of the ending node
    """
    def __init__(self, nodes, edges, cost, start_node = None, end_node = None):
        self.nodes = nodes
        self.edges = edges
        self.cost = cost
        if start_node is not None:
            self.start_node = start_node
        else:
            self.start_node = nodes[0]
        if end_node is not None:
            self.end_node = end_node
        else:
            self.end_node = nodes[-1]

        self.model = PLP.LpProblem("Shortest_Path", PLP.LpMinimize)
        # Non negative continous decision variable.
        self.x = PLP.LpVariable.dicts("x", self.edges, lowBound=0, cat = PLP.
        ↪ LpContinuous)

        # Objective
        if type(self.cost) is not dict:
            self.model += PLP.lpSum(self.cost[k] * self.x[e] for k, e in
            ↪ enumerate(self.edges)), "Objective"
        if type(self.cost) is dict:
            self.model += PLP.lpSum(self.cost[e] * self.x[e] for e in self.
            ↪ edges), "Objective"

    def construct_constraints(self):
```

```

        # One output from start node
        self.model += PLP.lpSum(self.x[e] for e in self.edges if e[0] == self.
↪start_node) == 1, "Start_Node_Constraint"
        # One input to end node
        self.model += PLP.lpSum(self.x[e] for e in self.edges if e[1] == self.
↪end_node) == 1, "End_Node_Constraint"
        # Balance constraint, inflow = outflow, for all nodes that are not start_
↪or end
        for node in self.nodes:
            if node != self.start_node and node != self.end_node:
                self.model += ((PLP.lpSum(self.x[e] for e in self.edges if e[0]
↪== node) -
                                PLP.lpSum(self.x[e] for e in self.edges if e[1]
↪== node)) == 0,
                                f"Balance_Constraint_{node}")

    def solve_and_print(self):
        self.model.solve()

        print("Status:", PLP.LpStatus[self.model.status])
        print("Objective value:", PLP.value(self.model.objective))
        for k, e in enumerate(self.edges):
            if self.x[e].varValue is not None and self.x[e].varValue > 0:
                print(f"Edge {e} is included in the solution with cost {self.
↪cost[k] if type(self.cost) is not dict
                    else self.cost[e]}")

```

This is a parent class of the clustering from shortest path defined below:

```

[60]: from modeller.network.shortest_path import *

class clustering_from_shortest_path(shortest_path):
    """
        This class implements a clustering algorithm based on shortest path from_
↪Uge 8 Klyingeinddeling Vha Korteste Veje
        Is uses the shortest_path class as a parent class.

        Input:
        Ascending ordered list of nodes.
        cost function that defines the cost of traveling from node i to node j.
        NOTE 1: Add 0 to the F since we need the Union of {0} and F
        NOTE 2: I MIGHT HAVE TO ADD 0 TO THE COST FUNCTION VALUES FOR ADDED START_
↪INDEX 0
    """
    def __init__(self, nodes, edges, cost_function):

```

```

start_node = nodes[0]
end_node = nodes[-1]
cost = cost_function(edges)
super().__init__(nodes, edges, cost, start_node, end_node)

def solve_and_print(self):
    self.model.solve()

    print("Status:", PLP.LpStatus[self.model.status])
    print("Objective value:", PLP.value(self.model.objective))
    for k, e in enumerate(self.edges):
        if self.x[e].varValue is not None and self.x[e].varValue > 0:
            print(f"Edge {e} is included in the solution with cost_
↪round(self.cost[k], 2)}")
            print(f"This corresponds to the group {self.nodes[e[0] + 1 :_
↪e[1] + 1 ]}\n")

```

I define the nodes in ascending order and a cost for each edge. We can be 3 to 5 people in each group.

```

[64]: #1)
group_sizes = [3,4,5]

F = "0.0 1.2 1.4 2.1 2.3 2.9 3.5 4.1 4.9 5.4 5.7 6.2 6.8"
F = F.split(" ")
F = [float(i) for i in F]
# 0 U Nodes
nodes = range(len(F))
# Possible edges are the ones where we jump within the possible groupsizes and_
↪the jump does not exceed the nodes.
edges = [(i, i+j) for i in nodes for j in group_sizes if i + j < len(nodes)]

def cost(edges):
    # Ordered handicap, so we can just compute the difference over the edges.
    return [F[j] - F[i + 1] for i, j in edges]

CFSP = clustering_from_shortest_path(nodes, edges, cost)
CFSP.construct_constraints()
CFSP.solve_and_print()

```

```

Status: Optimal
Objective value: 4.199999999999999
Edge (0, 4) is included in the solution with cost 1.1
This corresponds to the group range(1, 5)

Edge (4, 7) is included in the solution with cost 1.2
This corresponds to the group range(5, 8)

```

Edge (7, 12) is included in the solution with cost 1.9  
This corresponds to the group range(8, 13)

## 3.2 Spørgsmål 2

This is a min-max criteria. We wish to minimize

```
[69]: group_sizes = [3,4,5]

F = "0.0 1.2 1.4 2.1 2.3 2.9 3.5 4.1 4.9 5.4 5.7 6.2 6.8"
F = F.split(" ")
F = [float(i) for i in F]
# 0 U Nodes
nodes = range(len(F))
# Possible edges are the ones where we jump within the possible groupsizes and
↳ the jump does not exceed the nodes.
edges = [(i, i+j) for i in nodes for j in group_sizes if i + j < len(nodes)]

def cost(edges):
    # Ordered handicap, so we can just compute the difference over the edges.
    return [F[j] - F[i + 1] for i, j in edges]

CFSP = clustering_from_shortest_path(nodes, edges, cost)
minmaxcost = PLP.LpVariable("F", cat=PLP.LpContinuous, lowBound=0)
CFSP.model.setObjective(minmaxcost)
costs = cost(edges)
for i, j in edges:
    CFSP.model += costs[j] <= minmaxcost
CFSP.construct_constraints()
CFSP.solve_and_print()
```

Status: Optimal

Objective value: 2.6

Edge (0, 3) is included in the solution with cost 0.9

This corresponds to the group range(1, 4)

Edge (3, 6) is included in the solution with cost 1.2

This corresponds to the group range(4, 7)

Edge (6, 9) is included in the solution with cost 1.3

This corresponds to the group range(7, 10)

Edge (9, 12) is included in the solution with cost 1.1

This corresponds to the group range(10, 13)

### 3.3 Spørgsmål 3

Indsæt din besvarelse her.

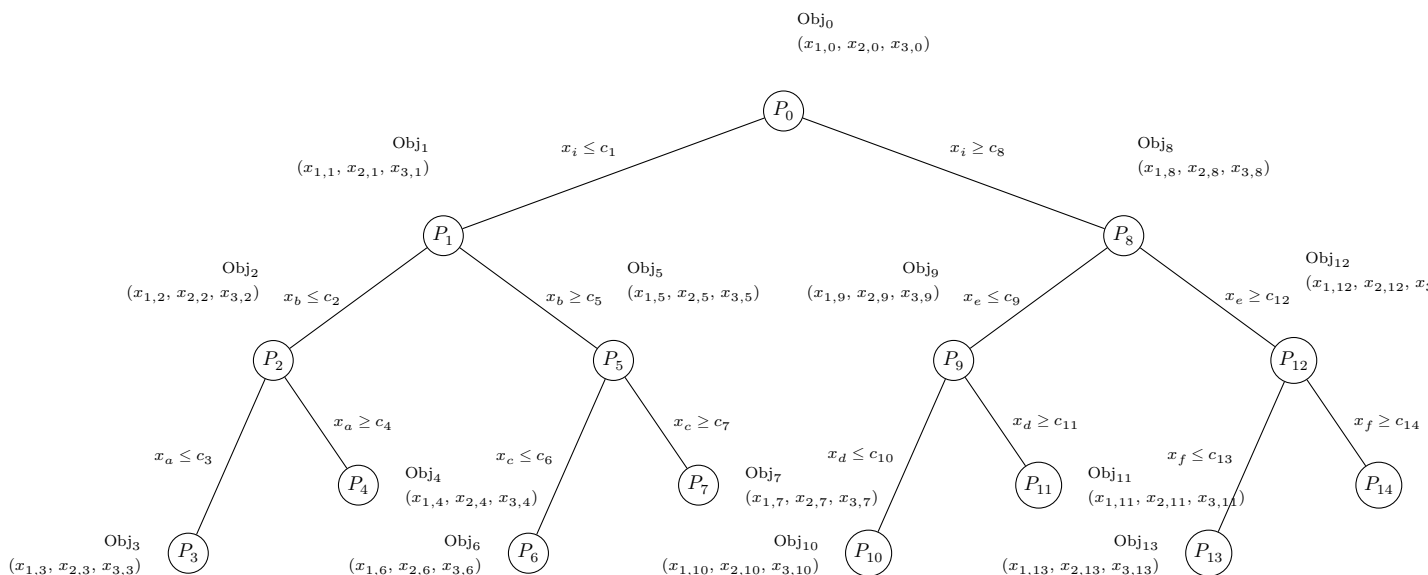
```
[ ]: %%sql
```

```
[30]: # Beregninger/kode til dette spørgsmål kan skrives her
```

### 3.4 Spørgsmål 4

Indsæt din besvarelse her.

```
[31]: # Beregninger/kode til dette spørgsmål kan skrives her
```



### 3.5 Spørgsmål 1

Indsæt din besvarelse her.

```
[32]: # Beregninger/kode til dette spørgsmål kan skrives her
```

### 3.6 Spørgsmål 2

Indsæt din besvarelse her.

```
[33]: # Beregninger/kode til dette spørgsmål kan skrives her
```

### 3.7 Spørgsmål 3

Indsæt din besvarelse her.

[34]: *# Beregninger/kode til dette spørgsmål kan skrives her*

### 3.8 Spørgsmål 4

*Indsæt din besvarelse her.*

[35]: *# Beregninger/kode til dette spørgsmål kan skrives her*